Phased Claude Code Build Prompts

Asset Allocation Recommender — Phase 1: Recommendation Engine → Phase 2: Chatbot-Based Rebalancing

How to use this document: Part 1 (next 5 pages) covers everything up to and including the allocation recommendation — no chatbot anywhere. Part 2 (final 3 pages) adds the chatbot and drift-triggered rebalancing on top of a working Phase 1. Paste each boxed prompt directly into Claude Code, alongside `CLAUDE.md` (the updated `claudeAsset.md`) at the repo root, and, for Module 2, the attached `forecaster.py` reference file.

Phase gate — do not skip this: nobody opens a Phase 2 branch until Phase 1 is merged to `main`, the full Phase 1 stack boots with one command, and someone outside the team can go intake → allocation → stock detail → MF detail without being told how. See `CLAUDE.md` §13 for the exact checklist.

#	Module	Phase 1	Phase 2	Directory
M1	Frontend	Intake form, dashboard, pie charts, KPI panels	ChatWindow, diff view, rebalance banner	/frontend
M2	Forecaster & Optimizer	Full scope — forecaster.py + optimizer.py	*None* — reused as-is	/backend/forecasting
M3	Backend/Schemas/Tax/Fundamentals	Core schemas, /profile, /allocate, /fundamentals, /mf-holdings, tax engine, fundamentals formatter	ChatIntent schema, /chat, /rebalance-check, profile_allocations store	/backend/api, /shared
M4	Chatbot + Universe Selection	**Only** universe_selector.py	Everything chatbot: ollama_client.py, intent_extractor.py	/backend/chatbot, /backend/selection
M5	Data Caching & Scraping	Full scope	*None* — same interface serves both phases	/backend/data

PART 1 OF 2

Phase 1 — Recommendation Engine

Capital in, allocation + pie charts + fundamentals out. No chatbot, no /chat, no /rebalance-check, no ChatIntent schema anywhere in this phase. Build and merge all five prompts below before touching Phase 2.

MODULE 1 OF 5 — PHASE 1

Frontend (Phase 1 scope)

React 18 + Vite + TypeScript + Tailwind + Recharts — no chat UI yet

Reads first: CLAUDE.md §0.2, §4, §5, §8, §9 | **Depends on:** /shared/schemas (mocked until M3 ships) | **Merge order:** last in Phase 1

CLAUDE CODE PROMPT — PASTE AS-IS

```
You are building Module 1 — Frontend, Phase 1 only of a 5-module student capstone
project called "Asset Allocation Recommender." This phase does NOT include the chatbot
-- no ChatWindow, no chat state, no /chat calls. That is explicitly deferred to Phase
2 (see CLAUDE.md Section 0.2); do not build it now, even partially.

Before writing any code, read CLAUDE.md at the repo root in full -- Sections 0.2, 4,
5, 8, and 9 define exactly what this phase must build.

Stack: React 18 + Vite + TypeScript + Tailwind CSS + Recharts, in /frontend.

Build, in order:
1. Intake form screen matching the UserProfile schema (capital, instruments multi-
select, bank, sector/MF-category preference, risk thermometer slider 0-100, tax slab
radio 10/20/30, horizon years).
2. Allocation dashboard: FD/MF/Stocks pie chart (Recharts PieChart), post-tax
expected-return range, and a risk meter -- all rendered from AllocationExplanation,
fetched via POST /allocate.
3. Stock detail accordion: one StockKpiPanel per held ticker, sourced from GET /
fundamentals/{ticker}. Render the income_statement / balance_sheet / cash_flow /
valuation KpiCard[] groups as bordered cards using the badge_color / badge_label the
API already computed -- do NOT recompute thresholds in TypeScript. Put a pie chart of
InstrumentWeight[] above the accordion for the stocks bucket.
4. MF detail: MfHoldingsPie component rendering MfHoldingsBreakdown.holdings as a pie
chart, with source_url and fetched_at shown as a small caption underneath.
5. src/api/client.ts: a typed fetch wrapper covering ONLY the four Phase 1 endpoints
in CLAUDE.md Section 4.2 (/profile, /allocate, /fundamentals/{ticker}, /mf-holdings/
{scheme}). Do not stub /chat or /rebalance-check yet -- they don't exist this phase.
6. src/mocks/: realistic fixture JSON for every Phase 1 response shape, so the entire
UI is demoable end-to-end via `npm run dev -- --mock` with zero backend running.

Constraints: no localStorage/sessionStorage -- use React state/context only. No
business logic (badge thresholds, tax math, allocation math) in this module; that all
lives server-side (CLAUDE.md Section 5). Keep the visual design clean and professional
-- this is a finance dashboard for a viva demo, not a toy UI.

Definition of done: `npm run build` produces a deployable static bundle, and `npm run
dev -- --mock` lets a reviewer click through intake -> dashboard -> stock detail -> MF
detail, with zero chat UI present anywhere and zero backend running.
```

MODULE 2 OF 5 — PHASE 1

Forecaster & Optimizer (full scope)

Locked Transformer config + Black-Litterman — complete in Phase 1, zero Phase 2 work

Reads first: CLAUDE.md §7, §7.1 + attached forecaster.py | **Depends on:** nothing real — synthetic inputs only | **Merge order:** 2nd, parallel with M4's universe selector

CLAUDE CODE PROMPT — PASTE AS-IS

```
You are building Module 2 — Forecaster & Optimizer of a 5-module student capstone
project. This module is entirely Phase 1 -- once you finish it, you have no further
work in Phase 2 (CLAUDE.md Section 0.2 table). Read CLAUDE.md Sections 7 and 7.1 in
full before starting.

Location: /backend/forecasting/ -- this directory is yours exclusively; nothing else
touches it.

Build:
1. forecaster.py -- copy the attached reference implementation VERBATIM. Do not
rewrite it, do not "improve" the hyperparameters (TRADING_DAYS=63, SEQ_LEN=20,
EPOCHS=200, d_model=32, nhead=4, num_layers=1, plus the MU_BLEND_ALPHA,
MU_CLAMP_SHARPE, SIGMA_CLAMP_BAND, LOG_VAR_CLAMP guards) -- this file is a locked
reference config, not a starting point. Confirm forecast_universe() still derives a
per-instrument seed via abs(hash(name)) % 2**32; never regress to calling
forecast_instrument() with the default seed=0 inside a loop over multiple instruments.
2. optimizer.py -- Black-Litterman via PyPortfolioOpt's BlackLittermanModel.
optimize() receives an ALREADY tax-adjusted mu dict (computed by Module 3's
tax_engine.py) and raw price histories (to compute the covariance matrix internally
via PyPortfolioOpt's risk_models). Equilibrium prior Pi = delta*Sigma*w_mkt, views
(P,Q) from the post-tax forecasts, Omega from the forecaster's calibrated sigma. Solve
the constrained QP: maximize w'mu_BL - (delta/2)*w'Sigma_BL*w subject to sum(w)=1, the
risk-thermometer-derived FD floor/ceiling (FD_min = max(0, 70 - 0.7r)%, FD_max =
max(5, 90 - 0.85r)%), and sector/instrument constraints.

Your ONLY public interface, used by both Phase 1's /allocate AND (unchanged, in Phase
2) /chat's recompute:
def forecast_universe(price_histories: dict[str, pd.Series]) -> dict[str, tuple[float,
float]]: ...
def optimize(mu_post_tax: dict[str, float], price_histories: dict[str, pd.Series],
risk_thermometer: float, constraints: dict) -> tuple[AllocationExplanation,
list[InstrumentWeight]]: ...

Build this interface generally enough that Phase 2's chatbot can call optimize() again
later with a modified constraints dict (e.g. an excluded sector) without you having to
touch this module again -- that is the whole point of accepting `constraints: dict` as
a parameter now instead of hardcoding today's constraint set.

Testing: build tests/smoke_test.py using synthetic hash-seeded price series and a
synthetic mu_post_tax dict, so this module is 100% testable without any other module's
real code running. Target: full pipeline for a 15-instrument universe runs in under 2
minutes on an RTX 4050 6GB laptop or CPU fallback.

Definition of done: forecast_universe() and optimize() both pass their smoke tests
```

```
standalone, with zero imports from /backend/api, /backend/chatbot, /backend/selection,
or /backend/data. Once merged, you are finished for both phases.
```

MODULE 3 OF 5 — PHASE 1

Backend API, Schemas, Tax Engine & Fundamentals

Core schemas + /profile, /allocate, /fundamentals, /mf-holdings + tax_engine.py + fundamentals_formatter.py

Reads first: CLAUDE.md §0.2, §4, §7.1, §8 | **Depends on:** M2/M4/M5 (all mocked until real) | **Merge order:** 3rd

CLAUDE CODE PROMPT — PASTE AS-IS

```
You are building Module 3 — Backend API, Schemas, Tax Engine & Fundamentals
Formatting, Phase 1 only of a 5-module student capstone project. This is the
orchestration hub for Phase 1 -- read CLAUDE.md in full (especially Sections 0.2, 4,
7.1, and 8) before starting. Do NOT build /chat, /rebalance-check, the ChatIntent
schema, or the profile_allocations store yet -- all four are explicitly Phase 2
(CLAUDE.md Section 0.2) and do not exist in this phase's code.

Location: /backend/api/ (routers, tax_engine.py, fundamentals_formatter.py) and /
shared/schemas (which you own and circulate).

Your FIRST deliverable, before anything else, is /shared/schemas/*.py -- ONLY the
Phase 1 models from CLAUDE.md Section 4.1: UserProfile, AllocationExplanation,
InstrumentWeight, KpiCard, StockFundamentals, MfHoldingsBreakdown. Do not add
ChatIntent yet -- it gets added at the start of Phase 2, not before. Finalize and
circulate this file to the other module owners before they write real implementation
code.

Build, in order:
1. /shared/schemas -- as above (Phase 1 models only).
2. tax_engine.py -- pure functions implementing mu_post_tax = (1 - T_i)*mu_nominal -
pi: FD and Debt MF taxed at the user's marginal slab (10/20/30%), Equity STCG 20% flat
(<=12 months), Equity LTCG 12.5% flat (>12 months). No cumulative Rs.1.25L LTCG
exemption tracking. pi (CPI) is a passed-in parameter, never hardcoded.
3. fundamentals_formatter.py -- takes raw fundamentals data (from Module 5's
cache_manager.get_fundamentals(ticker)) and produces a StockFundamentals object:
compute the 26 ratios (Net Profit Margin, Gross Margin, EBITDA, Revenue Growth, Total
Revenue, Net Income, Operating Margin, Earnings Growth, Current Ratio, Quick Ratio, D/
E, Working Capital, Total Cash, Total Debt, ROE, ROA, Operating CF, Free CF, P/E, P/B,
EV/EBITDA, Dividend Yield, Beta, PEG, Revenue per Employee, Payout Ratio), applying
the thresholds in /shared/kpi_thresholds.json to produce badge_label/badge_color/
tooltip already baked into each KpiCard.
4. FastAPI routers implementing exactly four endpoints: POST /profile, POST /allocate,
GET /fundamentals/{ticker}, GET /mf-holdings/{scheme}. /allocate must implement the
exact 5-step orchestration in CLAUDE.md Section 7.1: Module 4's select_universe ->
Module 5's get_prices for each survivor -> Module 2's forecast_universe -> your own
tax_engine.apply_post_tax per instrument -> Module 2's optimize.

Until Modules 2/4/5 land, stub each with a clearly-marked mock function (# MOCK --
replace with Module N call) returning realistic fixture data matching the schemas
exactly, so this module is independently runnable and demoable from day one via
```

```
`uvicorn backend.api.main:app --reload`.

Also do: auto-export openapi.json for Module 1's type generation, enable CORS for the
Vite dev server, add GET /health, log every request/response pair at DEBUG level.

Design ahead for Phase 2 without building it: keep your /allocate orchestration logic
in a clearly separated internal function (not inlined into the route handler), because
Phase 2's /chat endpoint will need to call that exact same orchestration again with a
modified profile/constraints -- you want to reuse it, not duplicate it, when that day
comes.

Definition of done: the full Phase 1 mock API is runnable standalone, every response
validates against /shared/schemas, and swapping a mock function for a real Module
2/4/5 call requires no change to any route signature.
```

MODULE 4 OF 5 — PHASE 1

Universe Selector (chatbot deferred to Phase 2)

ONLY backend/selection/universe_selector.py this phase

Reads first: CLAUDE.md §0.2, §6.2 | **Depends on:** M5's cache (mocked) | **Merge order:** 2nd, parallel with M2

CLAUDE CODE PROMPT — PASTE AS-IS

```
You are building Module 4 — Universe Selector, Phase 1 only of a 5-module student
capstone project. Your other responsibility, the chatbot, is entirely Phase 2 and does
not start yet -- do not create /backend/chatbot/, ollama_client.py,
intent_extractor.py, or the ChatIntent schema in this phase. Read CLAUDE.md Sections
0.2 and 6.2 before starting.

Location: /backend/selection/universe_selector.py -- this is your only deliverable in
Phase 1.

Build:
select_universe(profile: UserProfile, candidate_pool: list[str]) -> list[str]: filter
the candidate pool (from Module 5's config/universe.yaml, 10-15 instruments for the
prototype) down to the user's chosen sector/MF category, then call Module 5's
cache_manager.get_prices() for each candidate, compute the pairwise return-correlation
matrix, and where rho > 0.85 (configurable) drop the weaker-fundamentals instrument of
each highly-correlated pair (use Module 3's fundamentals data or a simple historical
Sharpe as the tiebreaker). Return the surviving, de-duplicated list -- this is exactly
what Module 2's forecast_universe() will be called with next, so it needs to be
correct and stable before Phase 1 can be demoed.

Public interface (the only thing any other module calls this phase):
def select_universe(profile: UserProfile, candidate_pool: list[str]) -> list[str]: ...

Testing: build a synthetic correlated-price-series test (e.g. two near-identical
random walks plus one independent one) and assert the dedup logic keeps exactly one of
the correlated pair. Test against mocked cache_manager.get_prices() -- you do not need
Module 5's real cache running to develop or demo this.

Definition of done: select_universe() works standalone against synthetic/mocked price
data, with zero dependency on Modules 2, 3, or 5 actually running. This is a smaller
Phase 1 deliverable than most other modules by design -- your larger piece of work
(the chatbot) is Phase 2; use any spare time this phase to help test/review other
modules.
```

MODULE 5 OF 5 — PHASE 1

Data Caching (full scope)

Cache manager + scrapers — complete in Phase 1, zero Phase 2 work

Reads first: CLAUDE.md §6.1, §8 | **Depends on:** none (true leaf module) | **Merge order:** 1st (build this first)

CLAUDE CODE PROMPT — PASTE AS-IS

```
You are building Module 5 — Data Caching. This module is entirely Phase 1 -- once
merged, you have no further work in Phase 2 (the same cache_manager interface serves
both phases unchanged, per CLAUDE.md Section 0.2). Every other module depends on you,
directly or indirectly, so build and stabilize this first.

Location: /backend/data/ -- yours exclusively.

Build:
1. cache_manager.py -- the ONLY interface any other module may use to reach price/NAV/
fundamentals/holdings data: get_prices(ticker, force_refresh=False),
get_fundamentals(ticker), get_mf_holdings(scheme). Back it with SQLite or Parquet
under backend/data/cache/ (gitignored). Bulk historical data is pulled ONCE per
instrument, on first access; every subsequent call reads from cache, and only the T-1
latest-day snapshot is ever re-fetched, cached with a same-trading-day TTL.
2. scrapers/yfinance_stocks.py, scrapers/mfapi_mf.py -- wrap the existing free data
sources (yfinance for stock OHLCV, mfapi.in for MF NAV history).
3. scrapers/moneycontrol.py -- scrape each configured MF scheme's portfolio holdings
percentage breakdown from its MoneyControl scheme page. Respect robots.txt, add a
fixed delay between requests, and cache with a ~30-day TTL. Every scrape must store
source_url and fetched_at alongside the parsed holdings, exactly matching the
MfHoldingsBreakdown schema in CLAUDE.md Section 4.1.
4. config/universe.yaml -- the capped candidate-instrument list (10-15 total stocks +
MF schemes for the prototype). This is the ONLY place instrument count changes when
the project scales past 100 instruments later.
5. data/fd_rates.json -- static, version-controlled, single last_updated stamp.

Testing: ship a --use-cache-fixtures flag so Modules 2, 3, and 4 can develop and run
tests against pre-baked fixture data without hitting real yfinance/MoneyControl
endpoints on every test run.

Definition of done: every other module can call cache_manager.get_prices /
get_fundamentals / get_mf_holdings and get real, cached, dated, source-stamped data
back -- with a documented fixture mode for offline development. Once merged, this
module needs no changes for Phase 2.
```

PART 2 OF 2 — DO NOT START UNTIL PHASE 1 IS MERGED AND DEMOABLE

Phase 2 — Chatbot-Based Rebalancing

Adds the ability to talk to the recommendation and have it rebalance. Only three modules have work this phase — M2 and M5 are already finished. Build in this order: M4's chatbot and M3's /chat + /rebalance-check together (they're tightly coupled), then M1's chat UI last.

MODULE 4 OF 5 — PHASE 2

Chatbot (Natural Language)

Ollama function-calling + strict Pydantic-validated intent extraction

Reads first: CLAUDE.md §10 | **Depends on:** a working Phase 1 (real AllocationExplanation available) | **Merge order:** 1st in Phase 2, together with M3

CLAUDE CODE PROMPT — PASTE AS-IS

```
You are building Module 4 — Chatbot (Natural-Language Intent), Phase 2 of a 5-module
student capstone project. Phase 1 (which included your universe_selector.py) is
already merged and working. Read CLAUDE.md Section 10 in full before starting. The
core requirement is precision: a wrong-but-confident allocation change is worse than
the bot asking a follow-up question.

Location: /backend/chatbot/ -- new this phase, yours exclusively.

Build:
1. ollama_client.py -- wrapper calling http://localhost:11434/api/chat with
qwen2.5:1.5b-instruct primary / llama3.2:1b latency fallback, using Ollama's function-
calling mode with six tool definitions: adjust_allocation(from_instrument,
to_instrument, percent), exclude_sector(sector_name), set_risk_level(value),
set_horizon(years), set_tax_slab(bracket), recompute_allocation().
2. intent_extractor.py -- takes {profile_id, message}, calls ollama_client, validates
the output into the ChatIntent schema (add this schema to /shared/schemas now,
coordinating with Module 3 -- it did not exist in Phase 1). On validation failure,
retry once with a corrective re-prompt; on second failure, return
ChatIntent(function="clarify", confidence=0). Also route to clarify when confidence <
0.7 even if schema-valid.
3. HARD RULE: raw LLM text must never reach Module 3's execution layer -- only the
validated ChatIntent object crosses that boundary.

Public interface:
def extract_intent(profile_id: str, message: str) -> ChatIntent: ...

Coordinate directly with whoever is building Module 3's /chat endpoint this phase --
your extract_intent() and their execution layer are tightly coupled and should be
developed and tested together, unlike Phase 1's modules which were fully independent.

Testing: build a fixture set of 20+ example user messages (clear commands, ambiguous
ones, adversarial/contradictory ones like "zero risk but exclude FD") and assert each
resolves correctly or falls back to clarify. Report your Pydantic pass rate and
clarify-trigger rate in your module README.

Definition of done: extract_intent() reliably returns a validated ChatIntent for clear
commands and reliably falls back to clarify for ambiguous/adversarial ones, verified
against your fixture set.
```

MODULE 3 OF 5 — PHASE 2

/chat, /rebalance-check & Persistence

Executes chatbot intents, adds drift-check, adds profile_allocations store

Reads first: CLAUDE.md §10, §4.2 (Phase 2 rows) | **Depends on:** M4's extract_intent(), working Phase 1 /allocate | **Merge order:** 1st in Phase 2, together with M4

CLAUDE CODE PROMPT — PASTE AS-IS

```
You are extending Module 3 — Backend API for Phase 2. Phase 1's schemas, tax engine,
fundamentals formatter, and four core endpoints are already merged and working. Read
CLAUDE.md Section 10 and the Phase 2 rows of Section 4.2 before starting.

Location: /backend/api/ (same directory as Phase 1) and /shared/schemas.

Build, in order:
1. Add the ChatIntent schema to /shared/schemas (coordinate with Module 4 -- this is a
joint addition, not a unilateral one).
2. profile_allocations persistence: a simple SQLite table (profile_id,
allocation_json, computed_at) that records every AllocationExplanation your /allocate
orchestration produces, going forward. This did not exist in Phase 1 because Phase 1
only ever computed one allocation per profile and never needed to diff against a prior
one.
3. POST /chat: calls Module 4's extract_intent(profile_id, message) -> ChatIntent. If
the function is one of adjust_allocation / exclude_sector / set_risk_level /
set_horizon / set_tax_slab / recompute_allocation, apply the parameter change to the
stored profile/constraints and re-run the SAME internal orchestration function you
wrote in Phase 1 for /allocate (reuse it, do not duplicate it). Return the new
AllocationExplanation plus a computed before/after diff. If the function is "clarify",
return the clarifying message with no recompute.
4. GET /rebalance-check/{profile_id}: re-runs the forecast + optimize step against the
freshest cached data, compares the result to the most recent row in
profile_allocations for that profile_id, and returns {drifted: bool, delta_pct: float,
message: str}.

Definition of done: a chat message correctly triggers a recompute and returns a diff;
an ambiguous message returns a clarify response with no recompute; rebalance-check
correctly flags drift when the freshly computed allocation differs meaningfully from
the last stored one, and correctly reports no drift when it doesn't.
```

MODULE 1 OF 5 — PHASE 2

Frontend — Chat UI

ChatWindow, before/after diff view, rebalance notification banner

Reads first: CLAUDE.md §9, §10 | **Depends on:** M3's /chat and /rebalance-check being live | **Merge order:** last in Phase 2

CLAUDE CODE PROMPT — PASTE AS-IS

```
You are extending Module 1 — Frontend for Phase 2. Phase 1's intake form, allocation
dashboard, stock detail, and MF detail screens are already merged and working. Read
CLAUDE.md Sections 9 and 10 before starting.

Location: /frontend/src/components/ (same project as Phase 1).

Build:
1. ChatWindow component: message list + input box, calling POST /chat, added to the
existing allocation dashboard (not a separate page -- users should be able to see the
dashboard and chat with it at the same time).
2. Diff view: when /chat returns an updated AllocationExplanation, render a clear
before -> after comparison (e.g. old vs new FD/MF/Stocks percentages, old vs new
expected return range) rather than just replacing the dashboard silently.
3. Rebalance notification banner: poll or check GET /rebalance-check/{profile_id} at
an appropriate point (e.g. on dashboard load) and show a dismissible banner if drifted
is true, with the message text from the response.
4. Extend src/api/client.ts with the two new Phase 2 endpoints and the ChatIntent type
(generate from Module 3's updated OpenAPI spec, same process as Phase 1).
5. Add Phase 2 fixtures to src/mocks/ so the chat flow is demoable via `npm run dev --
--mock` too.

Definition of done: a reviewer can complete the full Phase 1 flow, then open the chat
panel, type a plain-English adjustment, and see the dashboard update with a visible
diff -- all without leaving the single dashboard view.
```